Práctica guiada de Análisis Sintáctico con DeSiGNAR

Implementación de un analizador LL(1) en C++17

Compiladores | 2026

Índice

1. Objetivo **2**

2. Reconocer expresiones **2**

3. Implementación guiada **2**

3.1. Paso 1: definir la gramática . . . 3
3.2. Paso 2: reconocer los tokens . . . 3
3.3. Paso 3: preparar el flujo de tokens . . . 3
3.4. Paso 4: analizar y mostrar el árbol . . . 3
3.5. Paso 5: comprobar el comportamiento . . . 3
3.6. Código base . . . 3

4. Actividad: agregar la resta **5**

4.1. Tareas . . . 5
4.2. Pruebas sugeridas . . . 6

5. Entrega **6**

1. Objetivo

En esta práctica implementarás, paso a paso, un analizador sintáctico de expresiones aritméticas con las clases `Grammar`, `Lexer` y `LLParser` de DeSiGNAR. Partirás de una versión pequeña y funcional, seguirás el recorrido desde el texto hasta el árbol de análisis y, al final, ampliarás el lenguaje con un operador nuevo.

La dificultad es fácil-media. Se espera que puedas leer funciones sencillas de C++, modificar producciones de una gramática y probar un ejecutable. No es necesario implementar desde cero los algoritmos de análisis: DeSiGNAR construye la tabla LL(1) y realiza el análisis.

Requisitos. Usa C++17 y configura el ejecutable para que encuentre los encabezados de `include/` y enlace con DeSiGNAR (`Designar::Designar`), siguiendo el proyecto base.

Crea el archivo `practica_02.cpp` y trabaja sobre él.

2. Reconocer expresiones

El programa reconocerá números enteros no negativos, suma, multiplicación y paréntesis. No calculará el resultado: debe decidir si la entrada cumple la gramática y mostrar su árbol de análisis. La multiplicación tendrá mayor precedencia que la suma.

Usaremos esta gramática:

$$
\begin{aligned}
E &\to T\ E' \\
E' &\to \texttt{PLUS}\ T\ E' \mid \varepsilon \\
T &\to F\ T' \\
T' &\to \texttt{TIMES}\ F\ T' \mid \varepsilon \\
F &\to \texttt{LPAREN}\ E\ \texttt{RPAREN} \mid \texttt{NUM}
\end{aligned}
$$

E reúne sumas, T reúne multiplicaciones y F representa un número o una expresión entre paréntesis. Esta separación establece la precedencia sin tener que agregar reglas especiales al parser.

Antes de programar. Identifica cuáles son los no terminales y cuáles los terminales. En `Grammar::Sequence`, una producción vacía se representa con `Grammar::Sequence()`, que corresponde a ε. Cada nombre de terminal de la gramática debe coincidir con el nombre de una regla del lexer. Con la guía funcionando, calcula FIRST de F y explica por qué las alternativas de F se distinguen mirando el primer token.

3. Implementación guiada

Sigue los pasos en orden y ejecuta el programa después de cada cambio importante. El listado completo al final reúne las funciones de cada paso; puedes copiarlo al inicio y recorrerlo con la guía, o construirlo por partes.

3.1. Paso 1: definir la gramática

Implementa `make_grammar()`:

1) fija `E` como símbolo inicial;
2) agrega cada producción de la gramática planteada usando `Grammar::Sequence`;
3) representa las alternativas vacías de *E'* y *T'* con una secuencia vacía.

Al crear `LLParser parser(grammar)`, DeSiGNAR construye la tabla predictiva. Si dos producciones requieren la misma celda de la tabla, la construcción falla porque la gramática no es LL(1).

3.2. Paso 2: reconocer los tokens

Completa `make_lexer()`. Registra reglas para números, los operadores `+` y `*`, paréntesis y espacios. Las barras invertidas de los patrones se escriben escapadas en las cadenas de C++; por ejemplo, el patrón del signo más se escribe en C++ como `"\\+"`. El lexer conserva los espacios como tokens `WS`, así que estos se deben filtrar antes de llamar al parser.

3.3. Paso 3: preparar el flujo de tokens

En `tokenize_without_spaces()`, tokeniza el texto y copia a un nuevo arreglo todos los tokens excepto `WS`. El lexer guarda el nombre del token y el lexema original: por ejemplo, el texto `42` produce un token llamado `NUM` cuyo lexema es `42`.

3.4. Paso 4: analizar y mostrar el árbol

En `main()`, crea la gramática y el lexer, construye el `LLParser`, tokeniza una expresión y llama a `parser.parse(tokens)`. Recorre el árbol desde la raíz con `print_tree()` y libera la memoria usando `ParseTreeNode::destroy_tree(tree)`.

3.5. Paso 5: comprobar el comportamiento

Ejecuta con `8 + 3 * (2 + 1)` y confirma que se genera un árbol. Después prueba `8 + * 2`: el lexer reconoce sus tokens, pero el parser debe rechazar su orden. En cambio, `8 & 2` debe fallar durante la tokenización, porque no hay una regla para `&`.

3.6. Código base

```cpp
#include <iostream>
#include <stdexcept>
#include <string>

#include <llparser.hpp>

using namespace Designar;

Grammar make_grammar()
{
    Grammar g;
    g.set_start_symbol("E");

    g.add_production("E", Grammar::Sequence({"T", "E'"}));
```

```
    g.add_production("E'", Grammar::Sequence({"PLUS", "T", "E'"}));
    g.add_production("E'", Grammar::Sequence());

    g.add_production("T", Grammar::Sequence({"F", "T'"}));
    g.add_production("T'", Grammar::Sequence({"TIMES", "F", "T'"}));
    g.add_production("T'", Grammar::Sequence());

    g.add_production("F",
                     Grammar::Sequence({"LPAREN", "E", "RPAREN"}));
    g.add_production("F", Grammar::Sequence({"NUM"}));
    return g;
}

Lexer make_lexer()
{
    Lexer lexer;
    lexer.add_token("NUM",
                    "(0|1|2|3|4|5|6|7|8|9)"
                    "(0|1|2|3|4|5|6|7|8|9)*");
    lexer.add_token("PLUS", "\\+");
    lexer.add_token("TIMES", "\\*");
    lexer.add_token("LPAREN", "\\(");
    lexer.add_token("RPAREN", "\\)");
    lexer.add_token("WS", " +");
    return lexer;
}

DynArray<Token> tokenize_without_spaces(const Lexer& lexer,
                                        const std::string& input)
{
    DynArray<Token> all = lexer.tokenize(input);
    DynArray<Token> tokens;

    for (nat_t i = 0; i < all.size(); ++i)
    {
        if (all[i].name != "WS")
        {
            tokens.append(all[i]);
        }
    }

    return tokens;
}

void print_tree(ParseTreeNode* node, nat_t depth = 0)
{
    for (nat_t i = 0; i < depth; ++i)
    {
        std::cout << "  ";
    }

    std::cout << node->get_key().symbol;
    if (!node->get_key().lexeme.empty())
    {
```

```
69          std::cout << " (\"" << node->get_key().lexeme << "\")";
70      }
71      std::cout << '\n';
72
73      ParseTreeNode* child = node->get_first_child();
74      while (child != nullptr)
75      {
76          print_tree(child, depth + 1);
77          child = child->get_right_sibling();
78      }
79  }
80
81  int main()
82  {
83      Grammar grammar = make_grammar();
84      LLParser parser(grammar);
85      Lexer lexer = make_lexer();
86
87      const std::string input = "8 + 3 * (2 + 1)";
88
89      try
90      {
91          DynArray<Token> tokens = tokenize_without_spaces(lexer, input);
92          ParseTreeNode* tree = parser.parse(tokens);
93
94          std::cout << "Entrada: " << input << "\n\n";
95          std::cout << "Arbol de analisis:\n";
96          print_tree(tree);
97          ParseTreeNode::destroy_tree(tree);
98      }
99      catch (const std::runtime_error& error)
100     {
101         std::cerr << "Error: " << error.what() << '\n';
102         return 1;
103     }
104
105     return 0;
106 }
```

4. Actividad: agregar la resta

Amplía el analizador para aceptar la resta con la misma precedencia que la suma. La multiplicación debe conservar su mayor precedencia. No cambies `LLParser`: el objetivo es extender correctamente la gramática y mantener sincronizados los nombres de tokens.

4.1. Tareas

1) Agrega una regla léxica `MINUS` que reconozca `-`.
2) Agrega a E' una alternativa que consuma `MINUS`, T y después E'.
3) Prueba expresiones válidas con resta, suma, multiplicación y paréntesis. Comprueba que `9 - 4 * 2` se analiza sin error.

4) Prueba una entrada mal formada, por ejemplo `9 - * 2`, y confirma que el análisis sintáctico la rechaza.

5) Anota qué conjuntos FIRST y FOLLOW cambian y qué terminales pueden iniciar una continuación de *E'*.

Pista. La nueva alternativa de *E'* empieza por `MINUS`, mientras que la alternativa existente empieza por `PLUS`. Como esos terminales son distintos, el parser puede elegir entre las alternativas usando el lookahead. La producción vacía de *E'* no se modifica.

4.2. Pruebas sugeridas

Registra el resultado de cada entrada y clasifica cualquier fallo como léxico o sintáctico:

`Entrada`	Resultado
`42`	Aceptada
`(8 + 3) * 2`	Aceptada
`9 - 4 * 2`	Aceptada
`9 - * 2`	Rechazada
`(8 + 3`	Rechazada
`8 & 2`	Rechazada

5. Entrega

- Entrega el código fuente actualizado

- Realice un video breve explicando su solución a la actividad y la diferencia entre el error léxico y el error sintáctico observado.

- Suba el video a youtube y añada el link en el encabezado del archivo (ej: youtube.com, José Castro)